```
"""
SemanticEngine — Silver Ratio grounded compiler + executor.

STACK (correct separation):
    SemanticEngine          compiles intent → UMA_IR
        ↓
    UMA_IR                  execution graph
        ↓
    UMAExecutor             drives kernel
        ↓
    UMAClient               physics (untouched)

ENGINE3 BRIDGE — Silver grounded:
    binary weight:  x = len(binary(s)) / len(binary(anchor))
    complex state:  z = x * (C1 + i√e) = x * ((1-√2) + i√e)
    E_integral:     E = |z|²/2 = x² * (3-2√2 + e) / 2
    dz_dt:          (E - E_prev) / dt
    friction:       walks down by step_weight/(1 + |dz_dt|/floor) each pass

The imaginary component √e is the architectural bridge between:
    π   (continuous spacetime geometry)
    δ_S (discrete Silver Ratio / Pell recursion)

Closure (Ω) when:
    friction    < 0.15
    |dz_dt|     < silver_dz_dt_floor  (Silver thermal noise floor)
    |z - z*|    < 2 × field_scale
"""

from __future__ import annotations
import math
import numpy as np
from dataclasses import dataclass, field
from typing import Any, Dict, List, Optional, Tuple

from uma.observations.base import Observation, GaussianObservation
from uma.semantic.ir import UMA_IR, IRNode
from uma.semantic.friction import SemanticFriction, SemanticFrictionConfig, Frict
from uma.semantic.constraints import ConstraintSet, EqualityConstraint
from uma.semantic.inarticulation import Inarticulator, NullInarticulator, Product
from uma.semantic.executor import UMAExecutor, ExecutorResult
from uma.semantic.constants import (
    INV_DELTA_S_SQ, DELTA_S, C1_SILVER, SQRT_E,
    engine3_complex_state, engine3_E,
```

```
    silver_E_target, silver_dz_dt_floor, silver_field_scale,
)


# ---------------------------------------------------------------------------
# Config
# ---------------------------------------------------------------------------

@dataclass
class SemanticEngineConfig:
    friction: SemanticFrictionConfig = field(default_factory=SemanticFrictionConf
    stop_on_closure: bool = True
    verbose: bool = False
    obs_every: int = 1
    constrain_every: int = 1

    @classmethod
    def from_uma_config(cls, cfg, dt: float = 0.04, **kwargs) -> "SemanticEngineC
        """Build all thresholds directly from UMA Config — Silver-grounded."""
        return cls(
            friction=SemanticFrictionConfig.from_uma_config(cfg, dt=dt),
            **kwargs,
        )


# ---------------------------------------------------------------------------
# Result
# ---------------------------------------------------------------------------

@dataclass
class SemanticEngineResult:
    is_closed: bool
    closure_node: Optional[str]
    friction_records: List[FrictionRecord]
    friction_summary: Dict[str, Any]
    final_posterior: Any
    z_final: np.ndarray
    production_metrics: ProductionMetrics
    nodes_executed: int
    ir: UMA_IR

    def summary(self) -> str:
        pm = self.production_metrics
        fs = self.friction_summary
        return (
            f"\nSemanticEngine Result\n"
            f"  IR:                   {self.ir.summary()}\n"
```

```
            f"  Nodes executed:       {self.nodes_executed}\n"
            f"  Stage:                {pm.stage}\n"
            f"  Closed (Ω):          {self.is_closed}\n"
            f"  Closure node:         {self.closure_node}\n"
            f"  Friction:             {fs.get('friction_final','?'):.4f}\n"
            f"  dz/dt final:          {fs.get('dz_dt_final','?'):.4e}\n"
            f"  dz/dt floor (Silver): {fs.get('dz_dt_floor_silver','?'):.4e}\n"
            f"  E_final:              {fs.get('E_final','?'):.6e}\n"
            f"  E_target (kT/δ_S²):   {fs.get('E_target_silver','?'):.6e}\n"
            f"  Semantic density:     {pm.semantic_density:.4f}\n"
            f"  Stability index:      {pm.stability_index:.4f}\n"
            f"  1/δ_S² =              {INV_DELTA_S_SQ:.6f}\n"
        )


# ---------------------------------------------------------------------------
# SemanticEngine — the compiler
# ---------------------------------------------------------------------------

class SemanticEngine:
    """
    Compiles semantic intent into UMA_IR and executes via UMAExecutor.
    All thresholds derived from the Silver Ratio.
    """

    def __init__(
        self,
        z_target: np.ndarray,
        config: Optional[SemanticEngineConfig] = None,
        constraints: Optional[List[EqualityConstraint]] = None,
        inarticulator: Optional[Inarticulator] = None,
    ):
        self.z_target = np.asarray(z_target, dtype=float)
        self.config = config or SemanticEngineConfig()
        self._user_constraints = constraints
        self.inarticulator = inarticulator or NullInarticulator()

    @classmethod
    def from_uma_config(
        cls,
        cfg,
        z_target: np.ndarray,
        dt: float = 0.04,
        **kwargs,
    ) -> "SemanticEngine":
        """
        Build engine with all thresholds derived from UMA Config.
```

```
    This is the preferred constructor — no magic numbers.
    """
    engine_cfg = SemanticEngineConfig.from_uma_config(cfg, dt=dt)
    return cls(z_target=z_target, config=engine_cfg, **kwargs)

def compile(
    self,
    n_steps: int,
    dt: float,
    observations: Optional[List[Tuple[Observation, np.ndarray]]],
    constraint_set: ConstraintSet,
) -> UMA_IR:
    ir = UMA_IR()
    ir.metadata = {"z_target": self.z_target, "n_steps": n_steps, "dt": dt}

    obs_cycle = len(observations) if observations else 0
    obs_idx = 0

    for step in range(n_steps):
        sid = f"s{step:04d}"
        ir.append(IRNode(f"{sid}_evolve", "evolve", {"dt": dt}))

        if observations and step % self.config.obs_every == 0:
            obs, y = observations[obs_idx % obs_cycle]
            obs_idx += 1
            ir.append(IRNode(f"{sid}_observe", "observe",
                             {"observation": obs, "y": y}))

        if step % self.config.constrain_every == 0:
            ir.append(IRNode(f"{sid}_constrain", "constrain",
                             {"constraint_set": constraint_set}))

        ir.append(IRNode(f"{sid}_ckpt", "checkpoint", {}))

    return ir

def run(
    self,
    client,
    n_steps: int = 100,
    dt: float = 0.04,
    observations: Optional[List[Tuple[Observation, np.ndarray]]] = None,
) -> SemanticEngineResult:
    if client.posterior_state is None:
        raise RuntimeError(
            "Initialize UMAClient before SemanticEngine.run(): "
            "call client.initialize(psi0) first."
```

```
            )

        dim = len(self.z_target)
        cs = ConstraintSet(
            self._user_constraints,
            max_iters=20, tol=1e-6,
        ) if self._user_constraints else ConstraintSet.default_semantic_constrain
            dim, self.z_target
        )

        ir = self.compile(n_steps, dt, observations, cs)
        if self.config.verbose:
            print(f"[SemanticEngine] {ir.summary()}")
            print(f"[SemanticEngine] 1/δ_S² = {INV_DELTA_S_SQ:.6f}  "
                  f"E_target = {silver_E_target(self.config.friction.kT):.4e}  "
                  f"dz/dt_floor = {self.config.friction.convergence_dz_dt:.4e}")

        friction = SemanticFriction(self.z_target, self.config.friction)
        executor = UMAExecutor(
            friction=friction,
            inarticulator=self.inarticulator,
            stop_on_closure=self.config.stop_on_closure,
            verbose=self.config.verbose,
        )
        ex: ExecutorResult = executor.run(ir, client)

        return SemanticEngineResult(
            is_closed=ex.is_closed,
            closure_node=ex.closure_node,
            friction_records=ex.friction_records,
            friction_summary=ex.friction_summary,
            final_posterior=ex.final_posterior,
            z_final=ex.z_final,
            production_metrics=ex.production_metrics,
            nodes_executed=ex.nodes_executed,
            ir=ir,
        )


# ---------------------------------------------------------------------------
# Engine3 bridge — Silver grounded
# ---------------------------------------------------------------------------

def tokenize_to_binary_weight(s: str, anchor: str = "dIse") -> float:
    """Engine3's binary mapping: len(binary(s)) / len(binary(anchor))."""
    binary = ''.join(format(ord(c), '08b') for c in s)
    a_bin  = ''.join(format(ord(c), '08b') for c in anchor)
```

```
    return len(binary) / max(len(a_bin), 1)


def string_to_observation(
    s: str,
    dim: int,
    anchor: str = "dIse",
    cfg=None,
) -> Tuple[GaussianObservation, np.ndarray]:
    """
    Engine3 → UMA bridge, Silver-grounded.

    Pipeline:
        s → x  (binary weight)
        x → z_complex = x * ((1-√2) + i√e)     ← Silver seed
        E = |z_complex|²/2                       ← semantic integral
        E → normalized to field scale            ← no overflow
        y = z_target_dir * E_normalized           ← field-unit observation

    Observation noise R = (E_target)·I  where E_target = kT/δ_S².
    """
    x = tokenize_to_binary_weight(s, anchor)

    # Silver-grounded complex state
    z_c  = engine3_complex_state(x)
    E_raw = engine3_E(x)    # = |z_c|²/2 = x²*(3-2√2+e)/2

    # Normalize E to field scale
    if cfg is not None:
        kT      = cfg.generic.kT
        N       = cfg.grid.N
        rho_star = cfg.generic.mu / cfg.generic.g
        E_tgt   = silver_E_target(kT)
        fs      = silver_field_scale(kT, N, rho_star)
        # x_norm: sigmoid squash of binary weight to (0,1)
        x_norm  = x / (x + 1.0)
        y_scale = fs * x_norm
    else:
        # Fallback: use raw scale ratio (E_raw / (1 + E_raw))
        x_norm  = x / (x + 1.0)
        y_scale = x_norm * math.sqrt(2.0 * E_raw)

    y = np.full(dim, y_scale / max(dim, 1) * math.sqrt(dim))

    # Noise proportional to E_target (Silver scale)
    noise_var = max(y_scale**2 * INV_DELTA_S_SQ, 1e-10)
    R = noise_var * np.eye(dim)
```

```
obs = GaussianObservation(
    output_dim=dim, R=R, H=np.eye(dim),
    h_fn=lambda z: z, h_jac=lambda z: np.eye(len(z))
)
return obs, y
```